

Tehnici de căutare NAS

Stanciu Răducu-Ștefan

UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚĂ ȘI TEHNOLOGIE POLITEHNICA BUCUREȘTI
FACULTATEA DE ELECTRONICĂ, TELECOMUNICAȚII ȘI TEHNOLOGIA INFORMAȚIEI
PROGRAMUL DE MASTERAT: INGINERIA INFORMAȚIEI ȘI A SISTEMELOR DE CALCUL

Abstract

Prezenta lucrare investighează utilizarea conceptului de căutare NAS (Neural Architecture Search) pentru rezolvarea problemelor de optimizare a arhitecturii rețelelor neuronale artificiale. Provocarea principală abordată constă în complexitatea selecției manuale a hiperparametrilor, un proces costisitor din punct de vedere computațional și adesea suboptim. Metodologia propusă utilizează tehnologia Optuna pentru a evolua structurile unor rețele de tip HVTINY, VIT și LB-CNN, variind în mod dinamic parametrii specifici fiecărei rețele. Rezultatele experimentale sunt obținute pe seturile de date Cifar-10 și MNIST. Acestea evidențiază faptul că, printr-o căutare inteligentă a arhitecturii, este posibilă obținerea unor modele cu performanțe competitive, chiar și în condiții de resurse limitate. Studiul subliniază importanța automatizării procesului de proiectare a rețelelor neuronale și deschide noi perspective pentru cercetarea în domeniul inteligenței artificiale.

Cuvinte cheie: HVTINY, NAS, VIT, LB-CNN, OPTUNA, JAX.

1 INTRODUCERE

Evoluția rapidă a Internetului Lucrurilor (IoT) a generat o nevoie stringentă de a procesa datele direct la sursă, concept cunoscut sub numele de Edge AI. În acest context, TinyML reprezintă o barieră tehnologică ambițioasă, având ca scop rularea algoritmilor complecși de învățare automată pe dispozitive cu resurse extrem de limitate. Provocarea principală nu constă doar în obținerea unei acurateți ridicate, ci în echilibrarea strictă între performanța computațională, consumul de energie și memoria disponibilă (RAM și FLASH), care adesea nu depășește câteva sute de kiloocteți.

În mod tradițional, proiectarea arhitecturilor de rețele neuronale pentru astfel de dispozitive era un proces manual, laborios și care necesita o expertiză vastă. Pentru a depăși această limitare, acest proiect utilizează Neural Architecture Search (NAS) prin intermediul bibliotecii Optuna. Această abordare automatizează explorarea spațiului de căutare, permițând identificarea unor configurații optime de hiperparametri și structuri macro-bloc care să maximizeze eficiența modelelor.

Astfel, procesul de design trece de la intuiția umană la o optimizare probabilistică riguroasă, capabilă să descopere soluții inovatoare adaptate specificului fiecărui set de date.

Scopul central al acestui studiu este compararea unor arhitecturi de ultimă generație, precum HVTINY, LB-CNN și modele bazate pe Vision Transformers (ViT). Modelul HVTINY, de exemplu, este analizat prin prisma capacității sale de a reduce numărul de operații MAC prin utilizarea convoluțiilor separabile și eliminarea conexiunilor reziduale, oferind un compromis ideal între viteză și precizie. Pe de altă parte, explorarea modelelor ViT în acest ecosistem restrâns testează limitele actuale ale mecanismelor de atenție pe hardware de tip "low-power".

Obiectivul final al proiectului este de a demonstra că, prin tehnici avansate de preprocesare, augmentare a datelor și căutare automată a arhitecturii, este posibilă implementarea unor soluții de diagnosticare sau clasificare cu o amprentă de memorie minimă. Rezultatele obținute urmăresc să valideze modelele care pot fi convertite cu succes în formate compresate, precum TensorFlow Lite, oferind o bază solidă pentru viitoarele aplicații de monitorizare autonomă și inteligență distribuită.

2 NEURAL ARCHITECTURE SEARCH (NAS)

Neural Architecture Search (NAS) reprezintă procesul de automatizare a proiectării rețelelor neuronale pentru sarcini specifice, fiind un pas esențial în evoluția învățării automate (AutoML). Această disciplină a cunoscut o expansiune fulminantă, cu peste 1000 de lucrări publicate doar în intervalul 2020-2021, depășind în performanță arhitecturile concepute manual pe seturi de date complexe precum ImageNet.

Cadrul general al NAS este definit de trei piloni principali: spațiul de căutare, strategia de căutare și strategia de evaluare a performanței. Spațiul de căutare definește setul de arhitecturi posibile, variind de la structuri simple de tip lanț până la spații complexe bazate pe celule (cell-based) sau ierarhii. Arhitecturile bazate pe celule sunt cele mai populare, deoarece permit transferul celulelor optime învățate pe seturi mici de date către sarcini mai ample prin simpla multiplicare a acestora.

Strategiile de căutare au evoluat de la metode care nu are acces la structura internă a modelului (de tip „black-box”), precum învățarea prin recompensă (Reinforcement Learning) și algoritmi evolutivi, către tehnici mult mai rapide în care toate arhitecturile candidate sunt evaluate într-o singură antrenare (numite „one-shot”). Metodele tradiționale necesitau resurse imense deoarece fiecare arhitectură era antrenată de la zero. În schimb, tehnicile one-shot, precum DARTS, utilizează o rețea unificată care partajează greutatea între toate arhitecturile candidate, permițând optimizarea structurii prin gradient descent într-un timp semnificativ mai scurt. Pentru a accelera procesul, cercetătorii utilizează diverse tehnici de predicție a performanței, cum ar fi extrapolarea curbilor de învățare sau proxy-uri cu cost zero, care estimează succesul unei rețele fără antrenare completă. De asemenea, domeniul a făcut progrese mari în direcția reproductibilității prin lansarea unor benchmark-uri tabulare, precum NAS-Bench-101, care oferă rezultate precalculate pentru evaluări instantanee.

Viitorul NAS se îndreaptă către automatizarea completă a întregului flux de deep learning (AutoDL), optimizarea simultană a arhitecturii și a hiperparametrilor (NAS + HPO) și extinderea către domenii diverse, de la recunoașterea vorbirii și procesarea limbajului natural până la prognoza meteo și predicția plierii proteinelor. [1]

3 TEHNOLOGII AVANSATE DE OPTIMIZARE ȘI CALCUL NUMERIC

Succesul implementării unor modele complexe de inteligență artificială pe dispozitive cu resurse limitate depinde în mod critic de instrumentele software utilizate pentru optimizarea și antrenarea acestora. În acest proiect, procesului de dezvoltare a fost construit în jurul a trei tehnologii de ultimă generație: Optuna, Keras 3 și JAX. Integrarea acestora a permis trecerea de la o abordare bazată pe încercări manuale la un flux de lucru automatizat, capabil să descopere arhitecturi neuronale extrem de eficiente pentru mediul TinyML.

3.1 Optuna: Căutare Inteligentă a Arhitecturii Neuronale

Optuna este un cadru software de generație următoare, conceput pentru optimizarea automată a hiperparametrilor, abordând dificultățile tot mai mari ale proiectelor de învățare automată. Acesta introduce principii de design inovatoare, printre care se numără un API flexibil construit în timpul rulării, strategii eficiente de eșantionare și tăiere, și o arhitectură versatilă, ușor de configurat.

Acest tip de API permite utilizatorilor să construiască dinamic spațiul de căutare a parametrilor, oferind o flexibilitate superioară față de cadrele tradiționale care necesită o definire statică prealabilă. Această abordare modulară face ca Optuna să fie compatibilă cu sintaxa obișnuită Python, facilitând

reprezentarea unor spații de parametri complexe și eterogene. Utilizatorii definesc o funcție obiectiv care primește un obiect „trial”, prin care hiperparametrii sunt generați dinamic în timpul rulării.

Eficiența Optuna este susținută de mecanisme avansate de eșantionare și tăiere (pruning). Software-ul include algoritmi de eșantionare independenți, precum Tree-structured Parzen Estimator (TPE), dar și metode relaționale ca CMA-ES, care analizează corelațiile dintre parametri. Funcționalitatea de tăiere permite oprirea prematură a încercărilor care nu promit rezultate bune, monitorizând valorile intermediare ale funcției obiectiv. Optuna utilizează o variantă a algoritmului Asynchronous Successive Halving (ASHA), care este optimizat pentru medii de calcul distribuite, accelerând semnificativ procesul de explorare.

Arhitectura sistemului este scalabilă și versatilă, putând fi utilizată atât pentru experimente ușoare pe o singură mașină, cât și pentru sarcini masive de calcul distribuit. Optuna permite utilizatorilor să schimbe mediul de stocare a datelor (backend-ul de stocare), suportând baze de date precum SQLite sau baze de date relaționale pentru execuții paralele pe mai multe noduri.

Eficiența și utilitatea Optuna au fost demonstrate prin rezultate experimentale superioare în diverse scenarii, de la competiții Kaggle de detecție a obiectelor până la optimizarea supercomputerelor. Disponibil sub licență MIT, acest software open-source continuă să evolueze prin interacțiunea cu comunitatea, oferind un instrument esențial pentru cercetători și practicienii din domeniul inteligenței artificiale. [2]

3.2 JAX: Bibliotecă pentru calcul numeric accelerat

JAX este un framework de calcul numeric de înaltă performanță, dezvoltat de Google, care aduce o abordare modernă asupra cercetării în machine learning. Acesta combină simplitatea interfeței NumPy cu puterea accelerării hardware și a diferențierii automate. Principalul său avantaj este capacitatea de a transforma codul Python și NumPy în versiuni optimizate pentru unități de procesare grafică (GPU) și unități de procesare tensorială (TPU), oferind o viteză de execuție superioară framework-urilor tradiționale.

Sub capotă, JAX utilizează compilatorul XLA (Accelerated Linear Algebra) pentru a fuziona operațiile matematice și a reduce overhead-ul memoriei. Tehnologia se bazează pe patru transformări funcționale esențiale: grad (pentru calcularea automată a gradientilor), jit (Just-In-Time compilation pentru accelerarea funcțiilor), vmap (pentru vectorizarea automată a operațiilor) și pmap (pentru paralelizarea ușoară pe mai multe dispozitive). Această arhitectură permite cercetătorilor să scrie algoritmi complecși într-un stil funcțional, fără a sacrifica performanța necesară modelelor de scară mare.

În ecosistemul NVIDIA, JAX beneficiază de optimizări specifice prin biblioteci precum cuDNN și NCCL, asigurând o scalabilitate liniară în mediile de calcul distribuit. Pentru

proiecte de tip TinyML sau NAS, JAX devine o tehnologie strategică, deoarece permite validarea rapidă a mii de arhitecturi neuronale prin compilarea lor eficientă. În concluzie, JAX nu este doar o alternativă la NumPy, ci un motor de calcul transformativ care redefinește limitele eficienței în antrenarea rețelelor neuronale profunde. [3]

3.3 Keras 3.0

Keras 3 reprezintă o rescriere completă a framework-ului, transformându-l dintr-o interfață specifică pentru TensorFlow într-un motor de tip multi-backend. Această tehnologie permite rularea aceluiași cod Keras pe trei motoare de calcul diferite: JAX, TensorFlow și PyTorch. Această versatilitate oferă dezvoltatorilor libertatea de a alege backend-ul care oferă cele mai bune performanțe pentru arhitectura lor specifică, obținând adesea creșteri de viteză semnificative fără a modifica codul sursă.

Un avantaj major al Keras 3 este interoperabilitatea deplină între ecosisteme. Un model creat în Keras poate fi instanțiat ca un modul nativ PyTorch, exportat ca un TensorFlow SavedModel sau utilizat ca o funcție fără stare în JAX. Această abordare elimină riscul de a rămâne blocat într-un singur framework și permite accesul la instrumentele specifice fiecărei platforme, cum ar fi infrastructura de antrenare pe TPU-uri a JAX sau metodele de scalare masivă din PyTorch.

Din punct de vedere tehnic, Keras 3 introduce spațiul de nume keras.ops, care oferă o implementare completă a API-ului NumPy, dar agnostică față de backend. Aceasta înseamnă că orice componentă personalizată (straturi, funcții de pierdere sau metrice) scrisă folosind aceste operații va funcționa identic pe toate cele trei motoare. De asemenea, framework-ul suportă pipeline-uri de date din orice sursă, fiind compatibil cu obiectele tf.data.Dataset, PyTorch DataLoader sau array-uri NumPy.

În contextul TinyML, Keras 3 facilitează tranziția de la antrenarea de înaltă performanță către implementarea pe dispozitive Edge. Modelele pot fi optimizate folosind viteza superioară a JAX, iar ulterior pot fi convertite cu ușurință în formatul TensorFlow Lite (TFLite) pentru a rula pe microcontrolere. Această strategie de a crește complexitatea progresiv permite utilizatorilor să înceapă cu fluxuri de lucru simple și să coboare la nivel de detaliu doar atunci când au nevoie de o personalizare avansată. [4]

4 MODELE DE REȚELE NEURONALE

4.1 HVTINY

HVTINY este o arhitectură compactă de rețea neuronală convoluțională (CNN) de tip TinyML, propusă de cercetătorii români Radu Dogaru, Ioana Dogaru și Robert-Cristian Tecaru. Acest model a fost dezvoltat special pentru a rula

pe platforme microcontroler (MCU) cu costuri și resurse extrem de reduse, cum este modulul ESP32-CAM, destinat aplicațiilor medicale portabile de screening timpuriu (de exemplu, clasificarea leziunilor cutanate).

Spre deosebire de modelele mai complexe din care se inspiră (cum ar fi VRES-CNN), structura HVTINY elimină complet conexiunile de tip rezidual. Această decizie arhitecturală strategică reduce de aproximativ două ori numărul de operații de înmulțire și acumulare (MAC), scăzând semnificativ latența și consumul de energie pe hardware-ul restrâns. În plus, convoluțiile standard din majoritatea blocurilor componente sunt înlocuite cu structuri de tip separable și depth-wise convolution.

Flexibilitatea și scalabilitatea modelului sunt asigurate de un set de hiperparametri critici care permit adaptarea granulară a complexității rețelei:

- **Hiperparametrul k:** Reprezintă unitatea de bază pentru numărul de filtre. Toate straturile convoluționale își derivă lățimea ca multiplu al acestui factor k, permițând scalarea întregii rețele printr-o singură valoare.
- **Structura nl_top și nl_bot:** Acești parametri definesc adâncimea modelului. nl_top determină numărul de straturi de convoluție din blocurile superioare (unde se extrag trăsăturile de bază), în timp ce nl_bot controlează numărul de straturi din blocurile inferioare, permițând un control precis asupra ierarhiei de învățare.
- **Parametrul profil:** Reprezintă un tuplu de multiplicare (de exemplu, [1, 2, 2, 1]) care dictează cum evoluează numărul de filtre de-a lungul celor patru macro-blocuri ale rețelei. Un profil precum cel menționat permite rețelei să își mărească capacitatea de procesare în zonele intermediare (multiplicatorul 4) pentru a capta detalii complexe, reducând-o apoi spre final pentru a economisi memorie.

Performanța rețelei HVTINY a fost evaluată riguros pe seturi de date medicale convertite la rezoluții mici, de cel mult 48×48 pixeli, pentru a respecta constrângerile de memorie. Pe setul binar mel_skin_48 (detecția melanoamelor), modelul cuantizat în format INT8 (pentru compresie maximă prin LiteRT) a obținut o acuratețe de validare de 92,57%, necesitând doar 174 KB de memorie RAM și 99,3 KB de FLASH, cu o latență de scanare de 681 ms. Mai impresionant, pe setul de date multclasă skin_cancer_balanced (cu 7 clase de diagnostic), HVTINY a oferit cel mai bun compromis din studiu, atingând o acuratețe de 98,14%. Fișierul final .tflite ocupă doar 51 KB, modelul încadrându-se ideal în limita hardware uzuală de 256 KB RAM și dovedind că diagnosticul inteligent pe Edge este fezabil fără platforme computaționale scumpe. [5]

4.2 ViT

Prezentarea modelului Vision Transformer (ViT) implementat în Keras oferă o perspectivă asupra modului în care arhitecturile de procesare a limbajului natural pot fi adaptate cu succes pentru domeniul viziunii computaționale. Spre deosebire de rețelele neuronale convoluționale (CNN) tradiționale, care procesează imaginile prin filtre locale glisante, ViT abordează imaginea ca pe o secvență de date, aplicând mecanisme de atenție globală pentru a capta relațiile complexe dintre diferite regiuni ale scenei.

Conceptul fundamental din spatele ViT constă în divizarea unei imagini de intrare în patch-uri (părți pătrate egale) de dimensiuni fixe. Aceste patch-uri sunt apoi aplatizate și proiectate liniar într-un spațiu vectorial, devenind echivalentul "token-urilor" dintr-un text. Pentru ca modelul să înțeleagă structura spațială, fiecărui vector i se adaugă un embedding de poziție, permițând mecanismului de Multi-Head Attention să identifice unde se află fiecare fragment în raport cu celelalte. Această arhitectură permite modelului să aibă acces la informațiile globale (de exemplu întreaga imagine) încă de la primul strat, o capacitate pe care CNN-urile o dobândesc doar după multe straturi de procesare.

Performanța și dimensiunea acestui model sunt controlate prin hiperparametri specifici, care determină granularitatea analizei și capacitatea de învățare a rețelei:

- **patch_size:** Reprezintă dimensiunea laturii fiecărui fragment de imagine (de exemplu, 6x6 pixeli). O dimensiune mai mică a patch-ului crește rezoluția analizei, dar mărește exponențial lungimea secvenței procesate.
- **projection_dim:** Dimensiunea vectorului în care este proiectat fiecare patch. Acesta dictează dimensiunea modelului și capacitatea de reprezentare a trăsăturilor.
- **num_heads:** Numărul de capete de atenție în mecanismul de Multi-Head Attention, permițând modelului să se concentreze simultan pe tipuri diferite de relații vizuale.
- **transformer_layers:** Numărul de blocuri Transformer stivuite. Mai multe straturi permit învățarea unor trăsături mai abstracte, dar cresc riscul de overfitting pe seturi de date mici.
- **mlp_head_units:** Dimensiunea straturilor dense din capul de clasificare final, care interpretează reprezentarea globală construită de Transformer pentru a decide categoria imaginii.

În implementarea Keras, se subliniază faptul că, deși ViT este extrem de puternic, acesta necesită seturi de date masive sau tehnici avansate de augmentare și regularizare (precum weight decay și dropout) pentru a depăși performanța CNN-urilor. De cele mai multe ori, modelul este antrenat pe milioane de imagini și apoi ajustat (fine-tuning) pentru

sarcini specifice, demonstrând o scalabilitate superioară arhitecturilor clasice atunci când resursele computaționale sunt generoase. [6]

4.3 LB-CNN

Modelul LB-CNN (Light Binary Convolutional Neural Network) reprezintă o soluție inovatoare de tip TinyML, concepută special pentru a oferi un echilibru optim între acuratetea recunoașterii vizuale și complexitatea computațională redusă. Dezvoltat pentru a funcționa pe platforme cu resurse extrem de limitate, precum microcontrolerele (MCU) de tip ESP32-CAM sau dispozitivele FPGA, acest model se bazează pe utilizarea nucleelor de convoluție binare. Această caracteristică permite înlocuirea operațiilor costisitoare de înmulțire cu simple adunări sau operații logice, reducând dramatic consumul de energie și necesarul de memorie. Arhitectura LB-CNN este structurată ca un module de extindere neliniar, format din până la trei macro-blocuri convoluționale. Particularitatea acestui model constă în faptul că nucleele binarizate (cu valori de -1 și 1) sunt generate aleatoriu și nu sunt antrenate prin metode clasice de back-propagation. În schimb, procesul de învățare se concentrează exclusiv pe un singur strat liniar de ieșire, numit Adaline (ELM0). Acesta este antrenat ultra-rapid folosind algoritmul ELM (Extreme Learning Machine), care utilizează algebra liniară pentru a găsi greutatea optime într-o singură epocă. Această abordare permite o viteză de antrenare de sute de ori mai mare decât cea a mediilor tradiționale, facilitând optimizarea în mod iterativ a nucleelor binare prin rularea a numeroase încercări (trial-uri) într-un timp foarte scurt. Performanța modelului LB-CNN este dictată de un set de hiperparametri specifici, care permit reglarea fină a capacității de procesare:

- **Numărul de macro-blocuri convoluționale:** Modelul poate include între unul și trei blocuri, fiecare conținând operații de convoluție, activare neliniară și max-pooling.
- **Multiplicatorii de filtre (Factorul de multiplicare):** Aceștia determină numărul de nuclee binare per strat. De exemplu, un set de hiperparametri precum [8, 4, 2] indică faptul că numărul de canale de intrare este multiplicat progresiv la fiecare nivel.
- **Tipul de convoluție:** Modelul poate folosi convoluții 2D clasice sau depth-wise convolutions, cele din urmă fiind considerate mai performante în combinație cu binarizarea.
- **Parametrii de pooling și stride:** Dimensiunea ferestrei de pooling și pasul (stride, uzual 2) controlează reducerea rezoluției imaginii între etape, influențând direct latența și factorul de expansiune al vectorului de trăsături.

- **Parametrul de regularizare (C):** Folosit în cadrul antrenamentului ELM pentru a controla stabilitatea greutăților stratului de ieșire

În testele practice, LB-CNN a demonstrat rezultate remarcabile, obținând acurateți de până la 93% în detecția melanoamelor și chiar 100% în probleme de recunoaștere facială (setul ORL), confirmând eficiența cuantizării binare pentru dispozitivele de tip Edge. Deși modelul nu atinge performanțele de vârf ale rețelelor convoluționale tradiționale, acesta oferă un compromis excelent pentru aplicațiile în care resursele sunt extrem de limitate, demonstrând că o abordare neconvențională bazată pe binarizare și antrenament rapid poate fi extrem de eficientă în contextul TinyML. [7]

5 IMPLEMENTARE

Implementarea proiectului a fost realizată utilizând limbajul Python, ales pentru setul variat de biblioteci dedicate inteligenței artificiale și cercetării științifice. Mediul de dezvoltare folosit a fost Kaggle, care oferă acces gratuit la resurse de calcul GPU și o platformă colaborativă pentru experimentare. O parte importantă a arhitecturii software este utilizarea Keras 3, care permite o flexibilitate prin suportul multi-backend. Pentru acest proiect, a fost selectat backend-ul JAX, beneficiind astfel de compilarea XLA (Accelerated Linear Algebra) pentru o execuție ultra-rapidă a operațiilor pe GPU, esențială în faza de căutare a arhitecturii optime.

În faza de preprocesare, s-a pus un accent deosebit pe calitatea datelor de intrare. Imaginile au fost supuse unei proceduri de standardizare Z-score, calculând media și deviația standard a întregului set de antrenare. Această metodă asigură faptul că trăsăturile au o medie zero și o varianță unitară, accelerând convergența modelelor și prevenind instabilitățile numerice în timpul antrenamentului, indiferent de dataset-ul ales (MNIST, CIFAR sau imagini medicale).

Arhitectura HVTINY a fost implementată cu un accent pe eficiența hardware. Designul elimină complet conexiunile de tip rezidual (skip connections) pentru a reduce de aproximativ două ori numărul de operații de înmulțire și acumulare (MAC), scăderile de latență fiind vitale pentru platformele de tip Edge. În locul convoluțiilor standard, modelul utilizează structuri de tip Depthwise Separable, care separă filtrarea spațială de combinarea canalelor. Flexibilitatea rețelei este dictată de hiperparametrii structurali: unitatea de bază k determină lățimea rețelei, în timp ce nl_top și nl_bot controlează adâncimea ierarhică a blocurilor. În plus, utilizarea unui profil de multiplicare (precum **[1, 4, 2, 1]**) permite o scalare inteligentă a numărului de filtre în funcție de complexitatea trăsăturilor extrase la fiecare nivel.

Pentru modelul LB-CNN, implementarea a fost adaptată pentru acest proiect și urmează o abordare mai diferită, axată pe viteză extremă. Acesta utilizează nuclee de convoluție

binare generate aleatoriu, care rămân fixe și nu necesită antrenament prin backpropagation. Procesul de învățare este limitat exclusiv la stratul liniar de ieșire, antrenat prin algoritmul Extreme Learning Machine (ELM). Această abordare permite optimizarea modelului într-o singură epocă, transformând procesul de instruire într-o problemă de algebră liniară rezolvată aproape instantaneu pe GPU.

În paralel, a fost implementat modelul Vision Transformer (ViT), adaptând mecanismele de procesare a limbajului pentru viziunea computațională. Imaginea este divizată într-o secvență de patch-uri de dimensiuni fixe, care sunt ulterior proiectate liniar în spațiul embedding-urilor. Pentru a păstra informația spațială, sunt adăugate embedding-uri de poziție, datele fiind apoi procesate de straturi de Multi-Head Attention. Acest mecanism permite modelului să identifice relații globale între diferite regiuni ale imaginii încă de la primele niveluri de procesare.

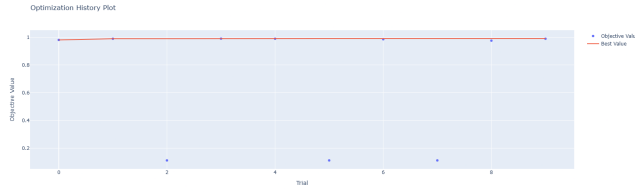
Pentru a găsi configurația ideală a fiecărei arhitecturi, s-a utilizat framework-ul de optimizare a hiperparametrilor Optuna. Fiecare model a fost încapsulat într-o funcție de tip **objective**, unde Optuna explorează spațiul de căutare pentru parametri precum rata de învățare, dropout-ul sau dimensiunea batch-ului, dar și parametrii specifici fiecărei arhitecturi. Pentru a maximiza eficiența timpului computațional pe Kaggle, s-au utilizat prunere (algoritmi de tăiere), care monitorizează performanța intermediară a fiecărui trial. Dacă un trial prezintă rezultate sub media celor anterioare, acesta este oprit prematur, permițând algoritmului să aloce resursele către configurații mai promițătoare.

6 REZULTATE EXPERIMENTALE

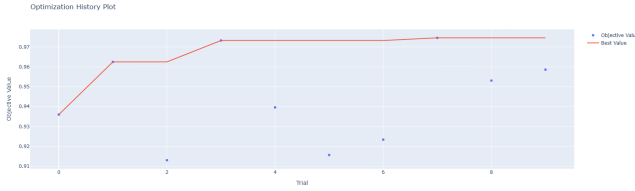
În această secțiune sunt prezentate performanțele obținute de cele trei arhitecturi analizate — HVTINY, Vision Transformer (ViT) și LB-CNN — utilizând seturi de date de referință încărcate direct de pe serverele TensorFlow. Procesul de evaluare a vizat nu doar precizia clasificării, ci și eficiența procesului de căutare a hiperparametrilor (NAS). Toate experimentele au fost rulate cu un număr de 10 epoci setate în cadrul funcției **objective** a Optuna, pentru a asigura o evaluare rapidă a diverselor configurații.

6.1 Analiza pe setul de date MNIST

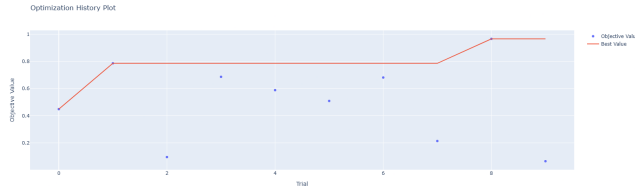
Setul de date MNIST, fiind un punct de referință standard și relativ simplu (imagini alb-negru ale cifrelor scrise de mână), a fost utilizat pentru a valida stabilitatea modelelor. Datorită complexității reduse, au fost necesare doar 10 trial-uri de optimizare, Optuna reușind să identifice configurații performante într-un timp extrem de scurt.



(a) Cautarea configurației optime pentru HVTINY



(b) Cautarea configurației optime pentru Vision Transformer



(c) Cautarea configurației optime pentru LB-CNN

Figure 1: Evoluția cautărilor configurațiilor optime

Criteriu	HVTINY	LB-CNN	ViT
Hiperparametri			
Structurali	$k = 25$ $nl_top = 1$ $nl_bot = 1$ $profil = [1, 4, 2, 1]$	$f1 = 12$ $f2 = 2$ $f3 = 8$ $conv5 = 1$ $nl1 = 1$ $nl2 = 2$ $flat = 11$	$patch_size = 8$ $projection_dim = 32$ $num_heads = 4$ $layers = 4$
Hiperparametri de Antrenare	$LR = 0.0017728$ $Batch = 128$ $Dropout = 0.4672$	$LR = 0.0016288$ $Batch = 32$	$LR = 0.0010187$ $Batch = 32$ $Dropout = 0.1588$
Acuratețe	99.40%	98.53%	98.06%
Țimp Antrenare [min]	1.64	5.67	2.53
Număr Parametri	13,010 (50.82 KB)	36,938 (144.29 KB)	23,638 (92.34 KB)

Table 1: Comparație între HVTINY, LB-CNN și Vision Transformer pe setul de date MNIST

Eficiența Timpului de Căutare:

- HVTINY: A ajuns la o configurație optimă în 10.57 minute, beneficiind de eliminarea conexiunilor reziduale care a redus numărul de operații per epocă.
- Vision Transformer (ViT): A necesitat 10.85 minute, un timp competitiv, însă ușor mai ridicat din cauza complexității mecanismului de Multi-Head Attention aplicat secvențelor de patch-uri.
- LB-CNN: A demonstrat cea mai mare viteză de execuție, finalizând cele 10 trial-uri în doar 2.71 minute. Acest lucru se datorează naturii sale binarizate și antrenamentului de tip ELM care nu necesită backpropagation pe straturile convoluționale.

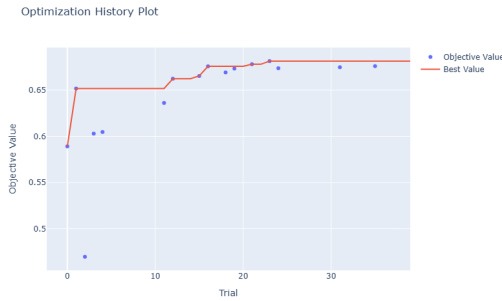
Toate cele trei modele au prezentat rezultate remarcabile pe acest set, atingând acurateți ridicate (peste 98%) cu un număr minim de parametri, demonstrând că până și modelele extrem de compacte pot rezolva probleme de clasificare standard cu resurse minime.

6.2 Analiza pe setul de date CIFAR-10

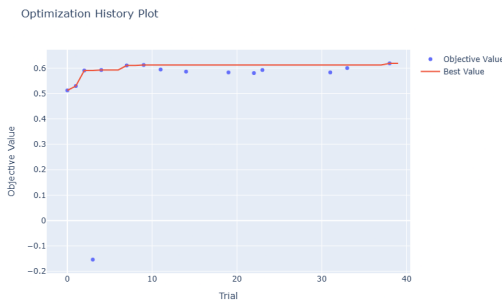
În cadrul acestei secțiuni, rezultatele obținute pe setul de date CIFAR-10 sunt analizate în detaliu, evidențiind provocările suplimentare aduse de imaginile color și diversitatea claselor de obiecte naturale. Tranziția de la MNIST la CIFAR-10 a impus o creștere a numărului de iterații în procesul de optimizare, fiind necesare 40 de încercări pentru ca algoritmul Optuna să poată identifica arhitecturile cele mai potrivite pentru complexitatea acestui dataset. Această extindere a spațiului de căutare a generat o creștere semnificativă a timpului total de procesare, reflectând efortul computațional necesar pentru a echilibra adâncimea rețelei cu precizia clasificării în contextul unor imagini cu rezoluție mică (32×32). Deși scenariul a fost mai solicitant, rezultatele finale subliniază eficiența arhitecturilor compacte, modelul care s-a detașat clar față de restul competitorilor fiind HVTINY. Acesta a demonstrat un echilibru superior între toți indicatorii de performanță urmăriti, reușind să ofere cea mai bună acuratețe de validare cu o amprentă de memorie extrem de redusă, confirmând astfel ipoteza cercetătorilor privind eliminarea conexiunilor reziduale pentru eficientizarea resurselor.



(a) Cautarea configurației optime pentru HVTINY



(b) Cautarea configurației optime pentru Vision Transformer



(c) Cautarea configurației optime pentru LB-CNN

Figure 2: Evoluția căutării configurațiilor optime

Mai jos sunt detaliate performanțele comparative ale celor trei modele:

- HVTINY: A reprezentat soluția optimă a studiului, obținând cea mai înaltă acuratețe pe CIFAR-10, în timp ce timpul de căutare prin Optuna a fost de aproximativ 37.38 minute. Deși este un model antrenabil, eficiența sa structurală i-a permis să se apropie de timpul de procesare al modelului binarizat, oferind în același timp un număr foarte mic de parametri, ideal pentru implementări pe microcontrolere.
- Vision Transformer (ViT): A necesitat cel mai mare consum de resurse și timp de căutare, finalizând procesul în 63.12 minute. În ciuda complexității sale și a utilizării

mecanismului de Multi-Head Attention, modelul a raportat o acuratețe inferioară celei obținute de HVTINY și un număr mult mai mare de parametri, confirmând dificultatea arhitecturilor de tip Transformer de a concura cu modelele convoluționale optimizate atunci când datele sunt limitate și rezoluția este scăzută.

- LB-CNN: S-a remarcat prin cel mai scurt timp de explorare, aproximativ 21.72 minute, datorită vitezei oferite de antrenamentul ELM și a greutăților binare. Totuși, din punctul de vedere al performanței, a prezentat o acuratețe mai slabă comparativ cu HVTINY, demonstrând că binarizarea extremă necesită o configurare mai complexă a filtrelor pentru a compensa pierderea de informație în cazul seturilor de date color.

Criteriu	HVTINY	LB-CNN	ViT
Hiperparametri			
Structurali	$k = 55$ $nl_{top} = 1$ $nl_{bot} = 1$ profil = [1, 4, 2, 1]	$f1 = 36$ $f2 = 98$ $f3 = 96$ convs = 1 $nl1 = 2$ $nl2 = 1$ flat = 1	patch_size = 2 projection_dim = 64 num_heads = 8 layers = 4
Hiperparametri de Antrenare	LR = 0.0050840 Batch = 64 Dropout = 0.2615	LR = 0.0008214 Batch = 32	LR = 0.0006665 Batch = 64 Dropout = 0.3759
Acuratețe	84.63%	61.67%	72.34%
Timp Antrenare [min]	11.18	6.95	44.88
Număr Parametri	54,350 (212.30 KB)	93,754 (366.23 KB)	151,882 (593.29 KB)

Table 2: Comparație între HVTINY, LB-CNN și Vision Transformer pe setul de date CIFAR-10

Sumarizând cele prezentate mai sus, analiza pe setul CIFAR-10 validează modelul HVTINY ca fiind cel mai performant model tehnologic. Acesta reușește să prezinte o complexitate mai scăzută chiar și față de modelele binarizate, de asemenea depășind și capacitatea de învățare a acestora și eficiența modelelor de tip Transformer, dovedindu-se soluția ideală pentru diagnosticul inteligent pe dispozitive cu resurse limitate. [8]

7 CONCLUZII

Rezultatele experimentale obținute pe seturile de date CIFAR-10 și MNIST confirmă faptul că HVTINY reprezintă cel mai performant „compromis” tehnologic dintre arhitecturile analizate. Acesta a dominat ambele scenarii, obținând acuratețea maximă de 84,63% pe CIFAR-10 și 99,40% pe MNIST, utilizând cel mai mic număr de parametri (respectiv 54.350 și 13.010). Modelul LB-CNN s-a remarcat prin simplitate, fiind cel mai rapid în antrenarea pe CIFAR-10 (6,95 min), însă acuratețea sa a scăzut semnificativ pe acest set complex, demonstrând limitele binarizării în raport cu modelele antrenabile. Pe de altă parte, Vision Transformer (ViT)

a necesitat resurse computaționale și timpi de antrenare mult mai mari, fără a egala eficiența modelelor convoluționale optimizate pe imagini de rezoluție mică. Astfel, prin eliminarea conexiunilor reziduale și utilizarea convoluțiilor depth-wise separable, HVTINY asigură o precizie ridicată și o memorie minimă ocupată, dovedindu-se soluția ideală pentru folosirea pe dispozitive de cu resurse limitate.

References

- [1] C. White *et al.*, “Neural architecture search: Insights from 1000 papers,” *arXiv preprint arXiv:2301.08727*, 2023.
- [2] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [3] “Nvidia jax release notes,” <https://docs.nvidia.com/deeplearning/frameworks/jax-release-notes/overview.html>, accesat la 16 mai 2026.
- [4] “Keras 3 documentation,” https://keras.io/keras_3/, accesat la 16 mai 2026.
- [5] R. Dogaru, D. Ioana, and R.-C. Tecaru, “Tinymml models for skin lesions classification,” in *Advances in Digital Health and Medical Bioengineering II: Volume 3: Telemedicine, Biomaterials, Environmental Protection, Medical Imaging, and Biomechanics*, 2026, p. 226.
- [6] “Image classification with vision transformer,” https://keras.io/examples/vision/image_classification_with_vision_transformer/, accesat la 19 mai 2026.
- [7] R. Dogaru and I. Dogaru, “Fast training of light binary convolutional neural networks using chainer and cupy,” in *2020 12th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)*. IEEE, 2020.
- [8] “Neural architecture search nas,” <https://github.com/raducu-stanciu/Neural-Architecture-Search-NAS-/blob/main/tehnici-de-cautare-nas.ipynb>.